

LambdaLens

王亦凡, 任睿欣, 杨倚
数信学院

April 21, 2026

1 摘要

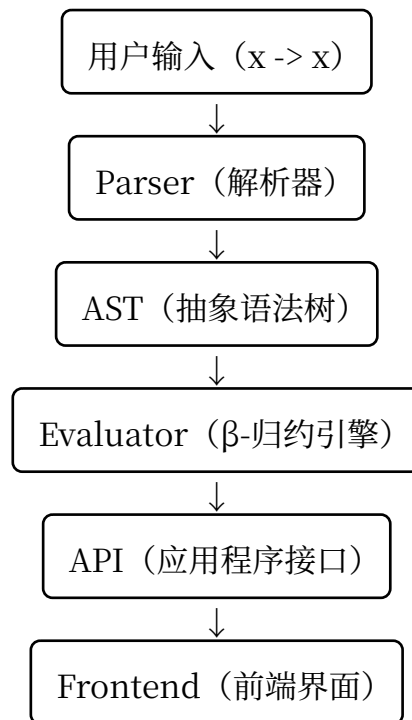
LambdaLens 是一个用于可视化 λ 演算, 逐步进行 β -规约过程的教学平台。项目采用了 HM 类型系统, 支持多种数据类型和函数定义, 且有良好的前端页面, 适合用于 λ 演算的教学与函数式编程的基本演示。

2 项目背景与意义

在编程语言的发展历程中, 函数式语言占据了重要的地位, 而 λ 演算作为函数式编程的理论基础, 对于理解函数式编程的核心概念至关重要。然而, λ 演算的抽象性常常使得初学者感到疑惑, 尤其是对于现代基于含类型 λ 演算的编程语言来说。因此, 开发一个能够可视化 λ 演算过程的教学平台, 可以有效帮助老师开展教学, 帮助学生更好的理解 λ 演算的核心概念和规约过程。不仅如此, 现代函数式编程语言的安装也是一些计算机基础薄弱的同学都难题, 开发一个在线的 Lambda 演算平台, 也能够帮助他们在不接触现代函数式编程语言繁重的工具链的情况下, 高效的学习函数式编程。并且, 国内互联网上对于 Lambda 演算的资料较少, 此项目也解决了现代 λ 演算教学资源匮乏的问题, 具有重要的教育意义和实用价值。

3 系统总体设计

3.1 系统架构图



3.2 模块说明

本系统采用分层模块化设计, 将 λ 演算表达式的解析、抽象语法树构建、 β -归约执行、可视化渲染与前端交互解耦, 形成清晰的技术结构。

3.2.1 Parser 模块

3.2.1.1 功能

负责将用户输入的 λ 演算表达式解析成抽象语法树 (AST)。支持基本的 λ 演算语法, 包括变量、函数抽象和函数应用。

3.2.1.2 内部实现

- 采用递归下降解析
- 支持变量、函数抽象、函数应用
- 处理括号优先级

3.2.2 AST 模块

3.2.2.1 功能

定义 λ 表达式的内部表示结构, 作为系统的核心中间层。

3.2.3 Evaluator 模块

3.2.3.1 功能

实现 β -规约引擎, 负责执行 λ 演算表达式的规约过程, 使用应用序列策略进行求值。接受 AST, 输出规约后的 AST。

3.2.3.2 内部逻辑

- 支持正常序 (Normal Order) 归约策略
- 实现 α -重命名避免变量捕获
- 实现替换 (Substitution) 算法

3.2.4 API 模块

3.2.4.1 功能

为前端提供统一的 HTTP 接口, 屏蔽内部实现细节。

提供接口包括:

- `/api/trace`: 返回 λ 演算表达式的规约过程的详细步骤。
- `/api/typecheck`: 提供类型检查服务, 验证 λ 表达式的类型正确性。
- `/api/eval`: 提供表达式求值服务, 执行 λ 演算表达式的规约。

3.2.4.2 设计理由

实现前后端分离, 便于维护和扩展, 同时也为未来可能的移动端或其他平台提供接口支持。

3.2.5 Frontend 模块

4 核心技术实现

4.1 λ 演算语法

系统采用 Haskell 风格的 λ 演算具体语法 (Concrete Syntax), 支持变量、函数抽象与函数应用。基本形式如下:

- 变量 (Variable): x
- 抽象 (Abstraction): $\lambda x \rightarrow e$
- 应用 (Application): $f\ x$
- 括号 (Parentheses): $f\ (g\ x)$ 用于显式指定优先级

4.2 AST 定义

```

data Expr
= EInt Int
| EBool Bool
| EVar String
| ELam String Expr
| EApp Expr Expr
| ELet String Expr Expr
| ELetRec String Expr Expr
| EIf Expr Expr Expr
| EBinOp Op Expr Expr
deriving (Show, Eq)

data Op
= Add
| Sub
| Mul
| Div
| Eq
| Lt
| Gt
deriving (Show, Eq)

```

本项目并没有选择较为简单的 AST 定义, 而是选择了一个较为复杂的 AST 定义, 支持整数、布尔值、变量、函数抽象、函数应用、let 表达式、递归函数定义、条件表达式以及二元操作。这是因为现代的 λ 演算教学不仅仅局限于纯粹的函数抽象和应用, 还需要涵盖更多的语言特性, 以便更好地模拟现代函数式编程语言的特性, 从而提高教学的实用性和针对性。

4.3 β -规约算法

因为现代函数式编程语言多为严格求值, 所以本项目采用了应用序列 (Applicative Order) 策略进行 β -规约。该策略先对函数和参数进行求值, 然后再进行函数应用。

本项目的 β -规约算法实现了以下功能:

- 支持应用序列 (Applicative Order) 策略进行规约
- 实现 α -重命名避免变量捕获
- 实现替换 (Substitution) 算法
- 支持递归函数的规约
- 支持条件表达式的规约
- 支持二元操作的规约
- 支持 let 表达式的规约
- 支持 letrec 表达式的规约
- 支持类型检查, 确保表达式的类型正确性

核心算法为: 替换 (Substitution), 可以用如下的 λ 演算表达式来表示:

```
( $\lambda x.$  e1) e2  $\rightarrow$  e1[x := e2]
```

4.4 HM 类型推导 (Algorithm W)

4.4.1 简述

系统实现了 Hindley-Milner (HM) 类型系统的核心算法 —— Algorithm W, 用于对 λ 表达式进行自动类型推导。HM 类型系统是现代函数式语言 (如 Haskell、OCaml、ML) 的理论基础。本系统的

HM 类型推导模块主要用于: 验证用户输入的 λ 表达式是否类型正确, 并在可能的情况下推导出其最泛类型。

HM 类型系统有如下特点:

- 无需类型注解即可自动推导类型
- 支持多态 (Let-Polymorphism)
- 能推导出表达式的最泛类型 (Principal Type)

4.4.2 类型系统定义:

```
data Type
  = TInt
  | TBool
  | TVar String
  | TFun Type Type
  deriving (Show, Eq)
```

系统支持以下类型构造:

- 基本类型: Int、Bool
- 类型变量: a、b、t1 等
- 函数类型: $\tau_1 \rightarrow \tau_2$

除以上静态类型, 系统也支持多态类型(Scheme):

```
data Scheme = Forall [String] Type
```

4.4.3 Substitutable 类型类

为了统一处理类型变量替换, 本系统实现了 Substitutable 类型类:

```
class Substitutable a where
  apply :: Subst -> a -> a
  ftv   :: a -> Set String
```

该类型类支持:

- 对 Type、Scheme、TypeEnv 进行替换
 - 计算自由类型变量 (Free Type Variables)
- 这是 Algorithm W 的基础设施。

4.4.4 替换 (Substitution) 与合一 (Unification)

系统实现了 Robinson 合一算法, 用于求解类型约束:

```
unify :: Type -> Type -> Either String Subst
```

支持:

- 基本类型匹配
- 类型变量绑定 (bind)
- 函数类型的递归合一
- 无限类型检查 (Occurs Check)

替换组合使用:

```
composeSubst :: Subst -> Subst -> Subst
```

用于合成多个替换, 确保类型推导过程中替换的一致性。

4.4.5 实例化与泛化

4.4.5.1 实例化 (Instantiation)

将多态类型 $\forall a. a \rightarrow a$ 转换为具体类型:

```
instantiate :: Scheme -> Infer Type
```

4.4.6 泛化 (Generalization)

在 `let x = e1 in e2` 中, 将 `e1` 的类型泛化为 Scheme:

```
generalize :: TypeEnv -> Type -> Scheme
```

```
x = \y -> y
```

对于上述表达式, 得到的泛化类型为:

```
x :  $\forall a. a \rightarrow a$ 
```

4.4.7 模块价值

HM 类型推导模块为系统提供:

- 静态类型检查能力
 - 自动类型推导能力
 - 多态支持 (Let-Polymorphism)
 - 递归函数类型推导 (LetRec)
 - 与 λ 演算归约引擎的深度结合
- 大幅度提升了系统的理论深度